

RoboPet — Phase 2, Steps 1 & 2

Firebase Wiring, Auth & the EconomySyncService

Goal. Stand up Firebase for the existing Flutter project, add anonymous Auth, extend PlayerProfile with the economy fields, and ship the production EconomySyncService that mirrors cloud economy into Isar (cloud wins) while pushing the care snapshot up (mirror only). After this, currencies are server-authoritative and ready for rollGacha.

1. Tooling setup (terminal, in order)

Prerequisites: Node.js >= 18 (node -v) and a working Flutter SDK (flutter doctor). Run everything from your project root unless noted. macOS/Linux shown; on Windows use PowerShell and the same commands.

1.1 Install the Firebase CLI & log in

```
# Install the Firebase CLI globally via npm
npm install -g firebase-tools
```

```
# Verify
firebase --version
```

```
# Authenticate (opens a browser)
firebase login
```

1.2 Activate the FlutterFire CLI

```
# Activate the FlutterFire CLI from pub
dart pub global activate flutterfire_cli
```

```
# Make sure the pub-cache bin is on your PATH (add to ~/.zshrc or ~/.bashrc)
export PATH="$PATH":"$HOME/.pub-cache/bin"
```

```
# Reload your shell, then verify
flutterfire --version
```

1.3 Add the Firebase packages to your Flutter project

```
# From your project root
flutter pub add firebase_core firebase_auth cloud_firestore cloud_functions
flutter pub get
```

1.4 Connect the project to Firebase

```
# Create a Firebase project first (or do it in the console), then:
flutterfire configure
# - Select (or create) your Firebase project
```

```
# - Select platforms: iOS + Android
# This generates lib/firebase_options.dart and registers the iOS/Android apps.
```

Platform minimums. In the Firebase console enable Authentication -> Anonymous and create a Cloud Firestore database (start in production mode; we add real rules below). For iOS, ensure ios/Podfile has platform :ios, 15.0. For Android, your minSdkVersion must be >= 21 (you already target API 26 / Android 8.0).

1.5 Firestore offline persistence

We rely on Isar for offline, but Firestore's own cache is a useful second line. It is enabled by default on iOS/Android; no action needed.

2. Initialize Firebase at app start

lib/main.dart (additions – keep your Phase 1.5 theme + Riverpod scope)

```
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'firebase_options.dart';
import 'data/isar_service.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  // 1) Firebase first.
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  // 2) Open Isar (Phase 1).
  final isar = await IsarService.open();

  runApp(
    ProviderScope(
      overrides: [
        isarProvider.overrideWithValue(isar),
      ],
      child: const RoboPetApp(),
    ),
  );
}
```

3. Authentication: anonymous-first

lib/data/auth/auth_service.dart

```
import 'package:firebase_auth/firebase_auth.dart';

/// Thin wrapper over FirebaseAuth. Anonymous-first; supports later linking.
```

```

class AuthService {
  AuthService(this._auth);
  final FirebaseAuth _auth;

  User? get currentUser => _auth.currentUser;
  Stream<User?> authStateChanges() => _auth.authStateChanges();

  /// Ensures we always have a uid. Safe to call on every cold start.
  Future<User> ensureSignedIn() async {
    final existing = _auth.currentUser;
    if (existing != null) return existing;
    final cred = await _auth.signInAnonymously();
    return cred.user!;
  }

  /// Upgrade the anonymous account to a permanent one (Phase 2.x).
  Future<UserCredential> linkCredential(AuthCredential credential) {
    final user = _auth.currentUser;
    if (user == null) {
      throw StateError('No anonymous user to link.');
```

4. PlayerProfile: economy fields + embedded currencies

We extend the Phase 1 PlayerProfile so it can hold the server-authoritative economy locally. playerId becomes the Firebase uid. economyVersion is the monotonic staleness guard from the TDD.

lib/data/models/player_profile.dart

```

import 'package:isar/isar.dart';

part 'player_profile.g.dart';

@collection
class PlayerProfile {
  Id id = Isar.autoIncrement;

  /// Firebase uid (replaces the Phase 1 'local-player' constant).
  @Index(unique: true, replace: true)
  late String playerId;

  String displayName = 'Operator';

  /// Monotonic counter mirrored from Firestore. The client NEVER increments
  /// this locally; it only adopts a value >= the one it already has.
  int economyVersion = 0;

  /// Embedded, server-authoritative balances.
  Currencies currencies = Currencies();

  /// Pity mirror (display only; server is authoritative).
  int epicPityCounter = 0;

  String? activeRobotId;

  int schemaVersion = 2;
}

@embedded
class Currencies {
```

```

int scrap = 0;      // soft currency (idle / care)
int rareCogs = 0;   // earned premium (Sudoku) -> Epic pulls
int premiumGems = 0; // earned-only in v1 (no IAP)
}

```

Migration note. Adding fields to an Isar collection is backward-compatible (new fields read as defaults), but you regenerated code, so re-run build_runner: `dart run build_runner build --delete-conflicting-outputs`

5. Cloud <-> local mapping

A small mapper keeps Firestore document shapes out of the rest of the app. It maps the meta/profile document into our embedded Currencies + scalar fields.

lib/data/sync/cloud_profile_mapper.dart

```

import 'player_profile.dart';

class CloudProfileMapper {
  /// Reads the Firestore `meta/profile` doc into a local PlayerProfile.
  /// Cloud is authoritative for economy, so we overwrite economy fields.
  static void applyEconomy(PlayerProfile target, Map<String, dynamic> data) {
    final c = (data['currencies'] as Map?) ?? const {};
    target.currencies
      ..scrap = _asInt(c['scrap'])
      ..rareCogs = _asInt(c['rareCogs'])
      ..premiumGems = _asInt(c['premiumGems']);
    target.epicPityCounter = _asInt((data['pity'] as Map?)?['epicCounter']);
    target.economyVersion = _asInt(data['economyVersion']);
    target.displayName = (data['displayName'] as String?) ?? target.displayName;
    target.activeRobotId = data['activeRobotId'] as String?;
  }

  /// Seed payload for a brand-new player (client-written ONCE on first run;
  /// allowed because the doc does not yet exist and contains no economy gains).
  static Map<String, dynamic> seedPayload(String uid) => {
    'playerId': uid,
    'displayName': 'Operator',
    'schemaVersion': 2,
    'economyVersion': 1,
    'currencies': {'scrap': 0, 'rareCogs': 0, 'premiumGems': 0},
    'pity': {'epicCounter': 0},
    'activeRobotId': null,
  };

  static int _asInt(Object? v) => (v as num?)?.toInt() ?? 0;
}

```

6. EconomySyncService (production)

This is the heart of Steps 1-2. It enforces the authority split: economy flows cloud->local (cloud wins, guarded by economyVersion); care flows local->cloud as a denormalized mirror only.

lib/data/sync/economy_sync_service.dart

```
import 'dart:async';

import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:isar/isar.dart';

import '../models/player_profile.dart';
import '../models/robot_instance.dart';
import 'cloud_profile_mapper.dart';

/// Owns the bidirectional, conflict-free sync between Isar and Firestore.
/// Economy: cloud -> local (cloud wins, version-guarded).
/// Care: local -> cloud (read-only mirror; disjoint field set).
class EconomySyncService {
  EconomySyncService({
    required Isar isar,
    required FirebaseFirestore db,
    required String uid,
  }) : _isar = isar,
       _db = db,
       _uid = uid;

  final Isar _isar;
  final FirebaseFirestore _db;
  final String _uid;

  DocumentReference<Map<String, dynamic>> get _profileRef =>
    _db.doc('users/$_uid/meta/profile');

  CollectionReference<Map<String, dynamic>> _sub(String name) =>
    _db.collection('users/$_uid/$name');

  // ---- Lifecycle -----

  /// Called once after auth resolves. Creates the profile if missing,
  /// then pulls cloud economy into Isar.
  Future<void> hydrateOnLogin() async {
    final snap = await _profileRef.get();
    if (!snap.exists) {
      await _seedNewPlayer();
    } else {
      await _applyCloudEconomy(snap.data()!);
    }
    await _hydrateModules();
  }

  /// Live economy subscription. Any server mutation (gacha, minigame reward)
  /// streams straight into Isar so the UI updates with zero extra plumbing.
  Stream<void> watchEconomy() =>
    _profileRef.snapshots().asyncMap((snap) async {
      final data = snap.data();
      if (data != null) await _applyCloudEconomy(data);
    });

  // ---- Cloud -> local (economy) -----

  Future<void> _applyCloudEconomy(Map<String, dynamic> data) async {
    final cloudVersion = (data['economyVersion'] as num?)?.toInt() ?? 0;
    await _isar.writeTxn(() async {
      var profile =
        await _isar.playerProfiles.filter().playerIdEqualTo(_uid).findFirst();
      profile ??= PlayerProfile()..playerId = _uid;

      // Staleness guard: never regress to an older economy snapshot.
      if (cloudVersion >= profile.economyVersion) {
        CloudProfileMapper.applyEconomy(profile, data);
        await _isar.playerProfiles.put(profile);
      }
    });
  }
}
```

```

    });
}

Future<void> _hydrateModules() async {
  final qs = await _sub('modules').get();
  await _isar.writeTxn(() async {
    for (final doc in qs.docs) {
      final d = doc.data();
      final existing = await _isar.moduleInstances
        .filter()
        .instanceIdEqualTo(doc.id)
        .findFirst();
      final m = existing ?? ModuleInstance()
        ..instanceId = doc.id
        ..defId = d['defId'] as String
        ..rarity = _rarityFrom(d['rarity'] as String?)
        ..level = (d['level'] as num?)?.toInt() ?? 1
        ..equippedRobotId = d['equippedRobotId'] as String?
        ..acquiredAt = _ts(d['acquiredAt'])
        ..schemaVersion = 2;
      await _isar.moduleInstances.put(m);
    }
  });
}

// ---- Local -> cloud (care mirror) -----

/// Push the read-only care snapshot. The security rules permit ONLY this
/// (careMirror + displayName), so it can never collide with economy writes.
Future<void> pushCareMirror(RobotInstance r) async {
  await _db.doc('users/${_uid}/robots/${r.instanceId}').set({
    'careMirror': {
      'battery': r.battery.value,
      'oil': r.oil.value,
      'entertainment': r.entertainment.value,
      'lastTickAt': r.lastTickAt.toUtc().toIso8601String(),
    },
  }, SetOptions(merge: true));
}

// ---- Seed -----

Future<void> _seedNewPlayer() async {
  // First-run write: doc doesn't exist yet, contains zeroed balances.
  await _profileRef.set(CloudProfileMapper.seedPayload(_uid));
  final snap = await _profileRef.get();
  await _applyCloudEconomy(snap.data());
}

// ---- helpers -----

static DateTime _ts(Object? v) {
  if (v is Timestamp) return v.toDate().toUtc();
  if (v is String) return DateTime.parse(v).toUtc();
  return DateTime.now().toUtc();
}

static ModuleRarity _rarityFrom(String? name) => ModuleRarity.values
  .firstWhere((e) => e.name == name, orElse: () => ModuleRarity.common);
}

```

7. Riverpod wiring

`lib/data/providers.dart` (new economy providers; keep Phase 1 isarProvider + repos)

```
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:isar/isar.dart';

import 'auth/auth_service.dart';
import 'sync/economy_sync_service.dart';
import 'models/player_profile.dart';

/// Provided via override in main() (Phase 1).
final isarProvider = Provider<Isar>((ref) => throw UnimplementedError());

final firebaseAuthProvider =
  Provider<FirebaseAuth>((ref) => FirebaseAuth.instance);
final firestoreProvider =
  Provider<FirebaseFirestore>((ref) => FirebaseFirestore.instance);

final authServiceProvider = Provider<AuthService>(
  (ref) => AuthService(ref.watch(firebaseAuthProvider)),
);

/// Ensures an anonymous session, returns the User (with uid).
final sessionProvider = FutureProvider<User>((ref) async {
  return ref.watch(authServiceProvider).ensureSignedIn();
});

/// Built once we have a uid; owns Isar<->Firestore sync.
final economySyncProvider = Provider<EconomySyncService>((ref) {
  final user = ref.watch(sessionProvider).requireValue;
  return EconomySyncService(
    isar: ref.watch(isarProvider),
    db: ref.watch(firestoreProvider),
    uid: user.uid,
  );
});

/// Kicks off hydration + keeps the economy stream alive for the app's life.
final economyBootstrapProvider = FutureProvider<void>((ref) async {
  final sync = ref.watch(economySyncProvider);
  await sync.hydrateOnLogin();
  final sub = sync.watchEconomy().listen((_) {});
  ref.onDispose(sub.cancel);
});

/// UI reads currencies from the single local source of truth (Isar).
final currenciesStreamProvider = StreamProvider<Currencies>((ref) {
  final isar = ref.watch(isarProvider);
  final user = ref.watch(sessionProvider).requireValue;
  return isar.playerProfiles
    .filter()
    .playerIdEqualTo(user.uid)
    .watch(fireImmediately: true)
    .map((rows) => rows.isEmpty ? Currencies() : rows.first.currencies);
});
```

How it composes: sessionProvider guarantees a uid -> economyBootstrapProvider hydrates Isar from cloud and subscribes to live economy changes -> the UI watches currenciesStreamProvider (pure Isar), so it works identically online or offline. Your Phase 1 care loop keeps writing Isar; call pushCareMirror after each reconcile to keep the cloud mirror fresh.

8. Firestore security rules

Deploy these so the authority split is enforced server-side. Save as `firestore.rules` and run `firebase deploy --only firestore:rules`.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{uid} {
      allow read: if request.auth != null && request.auth.uid == uid;

      // Profile: client may seed it once, and thereafter only touch
      // careMirror/displayName. Currencies & pity are CF-only.
      match /meta/profile {
        allow create: if request.auth.uid == uid;
        allow update: if request.auth.uid == uid
          && request.resource.data.diff(resource.data).affectedKeys()
            .hasOnly(['careMirror', 'displayName', 'activeRobotId']);
      }

      // Robots: client may write ONLY the careMirror field set.
      match /robots/{robotId} {
        allow read: if request.auth.uid == uid;
        allow write: if request.auth.uid == uid
          && request.resource.data.diff(resource.data).affectedKeys()
            .hasOnly(['careMirror']);
      }

      // Modules & pulls are granted/written by Cloud Functions only.
      match /modules/{moduleId} { allow read: if request.auth.uid == uid; }
      match /pulls/{pullId}      { allow read: if request.auth.uid == uid; }
    }
  }
}
```

9. Verification checklist

- [] flutterfire configure generated `lib/firebase_options.dart`; app launches with no Firebase init error.
- [] First cold start creates `users/{uid}/meta/profile` with zeroed currencies (check the Firestore console).
- [] `currenciesStreamProvider` shows 0/0/0 in the HUD; editing currencies in the console updates the HUD live (validates `watchEconomy`).
- [] Turning off the network still renders the last-known currencies from `Isar` (offline-first).
- [] After a care action, `users/{uid}/robots/{id}.careMirror` updates; attempting to write currencies from the client is rejected by rules.

Next (Steps 3-4): the `ModuleInstance` collection + inventory grid, then the `rollGacha` Cloud Function with the Remote Config loot table and the Dart `GachaService`.